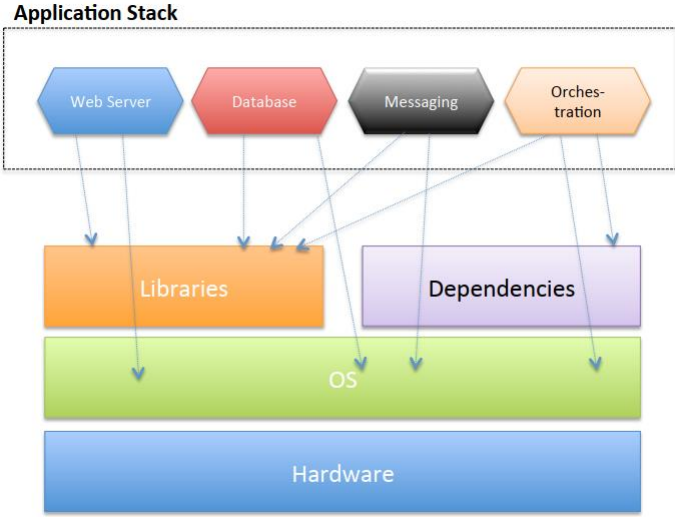
Chapitre 3 : Conteneurisation avec Docker

Dr. Ing. Sana REKIK

sana.rekik@istic.ucar.tn

Architecture traditionnelle



Problème de compatibilité / de dépendance

Il n'est pas facile d'assurer la compatibilité entre les différents composants de l'application et toutes les versions d'OS, bibliothèques…

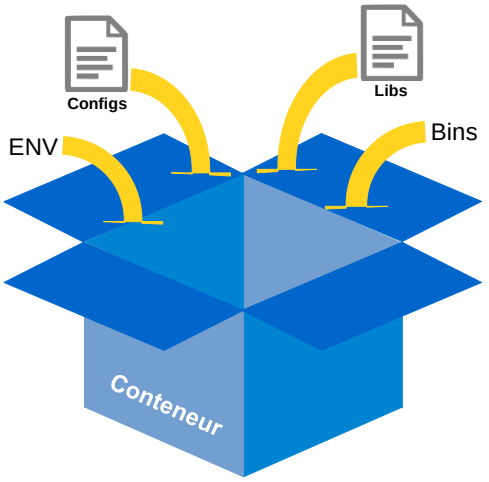
Différents environnements : développement, test, production

Longue durée d'installation

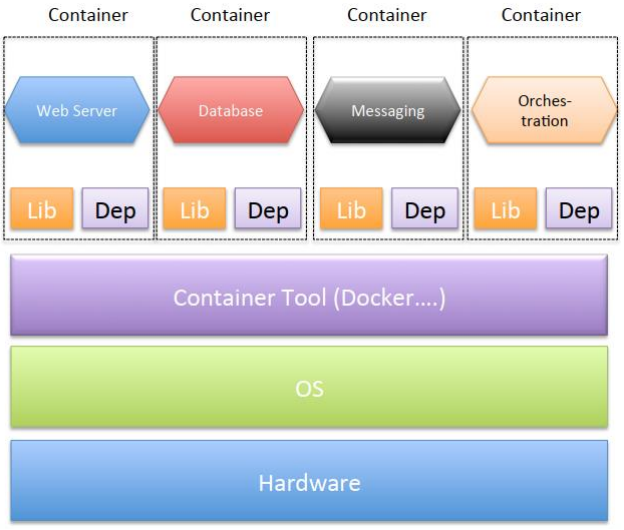
Pourquoi utiliser la conteneurisation ?

Conteneurisation élimine ces problèmes

- Regroupe le code d'une application avec toutes les bibliothèques et les dépendances nécessaires à son exécution dans un environnement d'exécution virtuel (Virtual Environment – VE) isolé appelé **conteneur**
  - Package portable : s'exécute sur n'importe quelle plateforme ou dans n'importe quel cloud sans problèmes



Pourquoi utiliser la conteneurisation ?

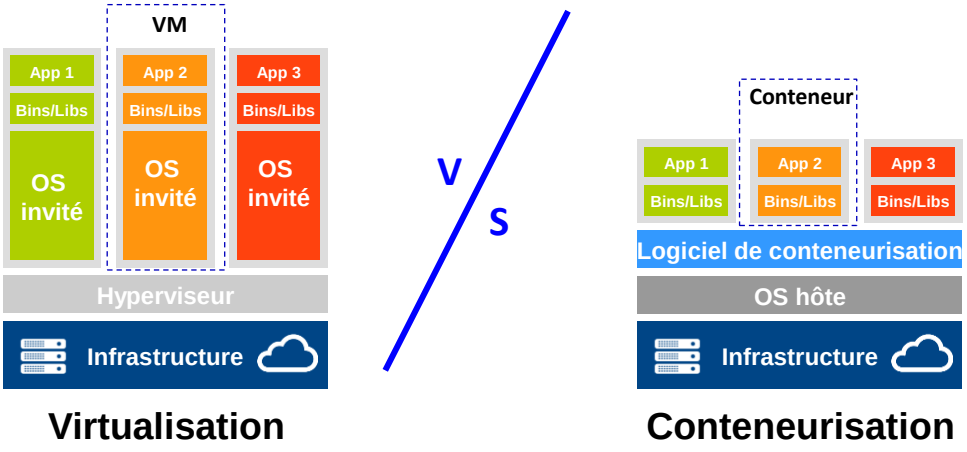


Exécuter chaque service avec ses propres dépendances dans des conteneurs séparés

La conteneurisation

- La conteneurisation = virtualisation par conteneurs : forme de virtualisation applicative
- Elle permet de créer un environnement indépendant qui rassemble tout ce qu'il faut pour que l'application fonctionne correctement, sans dépendre de ce qui est installé sur un environnement informatique spécifique
- Le conteneur s'exécute de manière native et **partage le noyau** de la machine hôte avec d'autres conteneurs. Il ne prend pas plus de mémoire que tout autre exécutable → ce qui le rend **léger**
- Les conteneurs :
  - Ne virtualisent pas les ressources matérielles, ils créent juste une isolation des applications
  - Ils partagent les ressources avec le système hôte, pas de réservation de ressources nécessaires (CPU, RAM et disque) auprès du système hôte
  - Démarrent très rapidement (plus rapide que les VMs)

Conteneurisation vs virtualisation



Virtualisation **Conteneurisation**

Un Conteneur Linux, qu'est-ce que c'est ?

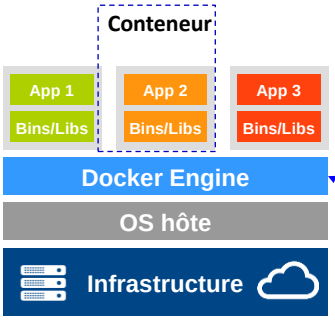
- Les conteneurs Linux **LXC** (Linux Container) existent depuis plus longtemps que **Docker**
- Un conteneur Linux est une enveloppe virtuelle qui permet de packager une application avec tous les éléments dont elle a besoin pour fonctionner :
  - fichiers source, runtime, librairies et dépendances
- Les conteneurs sont isolés du reste du système. Ils sont packagés en un ensemble cohérent et prêt à être déployé sur un serveur et son OS
  - Ils sont portables et fonctionnent de la même manière dans les environnements de développement, de test et de production.
  - Ils permettent de déplacer l'application jusqu'en production sans aucun effet secondaire.

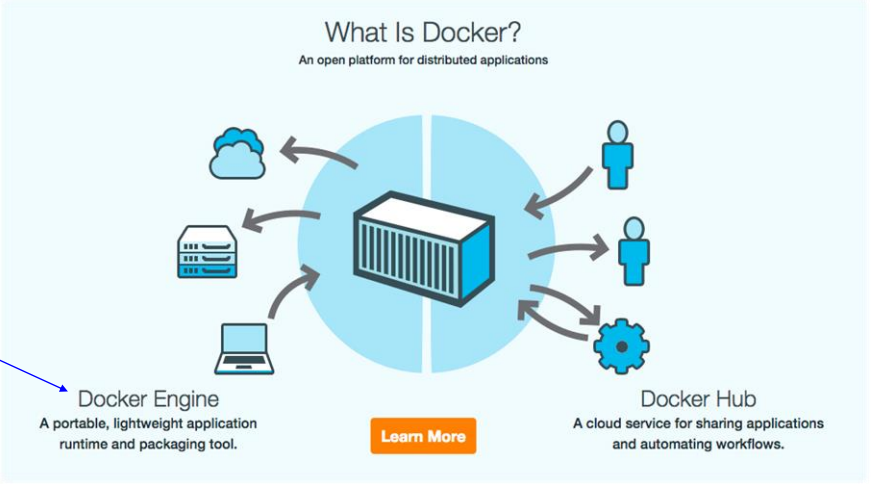
Avantages des conteneurs Linux

- Un conteneur Linux mobilise moins de ressources qu'une VM.
- Créer un conteneur est **plus rapide** que lancer une VM, mais une VM <u>offre une meilleure isolation.</u>
- Les images de conteneurs Linux permettent d'assurer la portabilité et le contrôle des versions des applications
  - Les développeurs ont ainsi la garantie que ce qui fonctionne sur leur ordinateur fonctionnera aussi dans l'environnement de production.
  - Cela permet également de pouvoir tester, sur la même machine, plusieurs versions d'un même logiciel.

Qu'est-ce que Docker ?

- Plateforme de conteneurisation open source la plus populaire et la plus utilisée de nos jours





Qu'est-ce que Docker ?

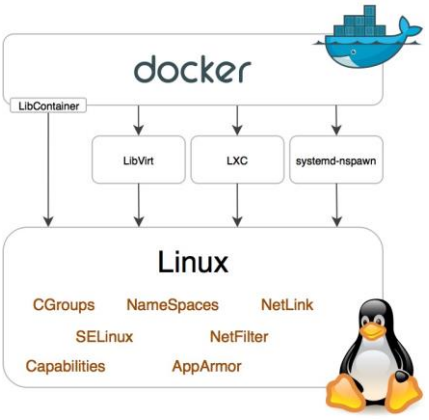
- Un projet ouvert développé par la société Dotcloud (renommée par la suite Docker) sur la base de conteneur Linux standard (LXC : Linux Container).
- Un outil qui peut empaqueter une application et ses dépendances dans un conteneur virtuel, qui pourra être exécuté sur n'importe quel serveur Linux.
- Il étend LXC et permet également de lancer des conteneurs sur les SE Windows et Mac.
- Docker peut être déployé sur le Cloud, raison pour laquelle de géants de ce domaine (tels que Google, Microsoft ou Amazon) ont décidé de l'implémenter au sein de leurs systèmes.

Build, Ship and Run
Any App, Anywhere

docker

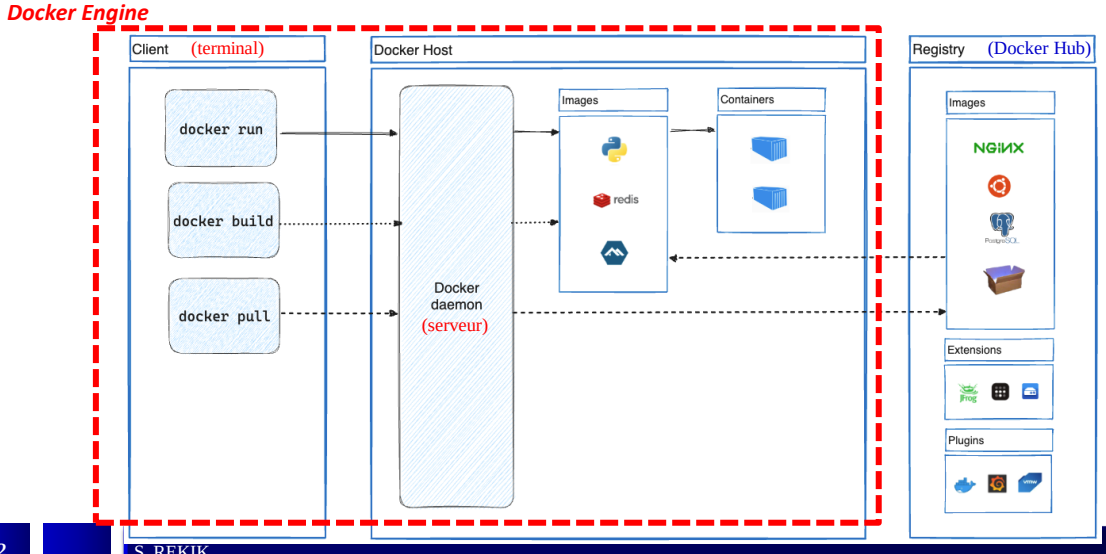
Qu'est-ce que Docker ?

- Depuis la version 0.9 (2014), Docker n'utilise plus LXC comme environnement d'exécution par défaut .
- Il l'a remplacé par sa propre bibliothèque **Libcontainer** , implémentée en Go, un langage de programmation orientée service
  - abstraction du système d'exploitation hôte.
- Cette bibliothèque présente un intérêt important dans le déploiement continue : elle permet l'exécution de Docker sur des systèmes d'exploitations ne se basant pas sur le noyau Linux (e.g. Windows, Mac, BSD, Solaris).



Architecture Docker

- Docker permet aux utilisateurs de démarrer, d'arrêter et de gérer les conteneurs logiciels directement à partir du terminal (CLI)



Docker Engine

- C'est le moteur Docker. C'est une application qui suit une architecture client-serveur comprenant principalement trois composants : **Docker Daemon**, **Docker CLI** et une **API** qui sert entre autres d'intermédiaire entre les deux composants précédents.

1. **Docker Daemon** : la partie serveur de Docker Engine
   - Il s'exécute en arrière-plan sur le système hôte et sert à contrôler le moteur Docker de manière centralisée. Il crée et gère toutes les images, conteneurs ou réseaux.
   - Il est adressé à l'aide de commandes comme build (créer), pull (télécharger) ou run (démarrer).
2. **API :** basée sur le paradigme de programmation REST
   - Utilisée pour indiquer au démon docker ce qu'il doit faire.
3. **Docker CLI** : interface utilisateur (client) qui interagit avec le daemon via l'API REST et permet aux utilisateurs de le contrôler grâce aux scripts ou aux entrées utilisateur.

Image Docker

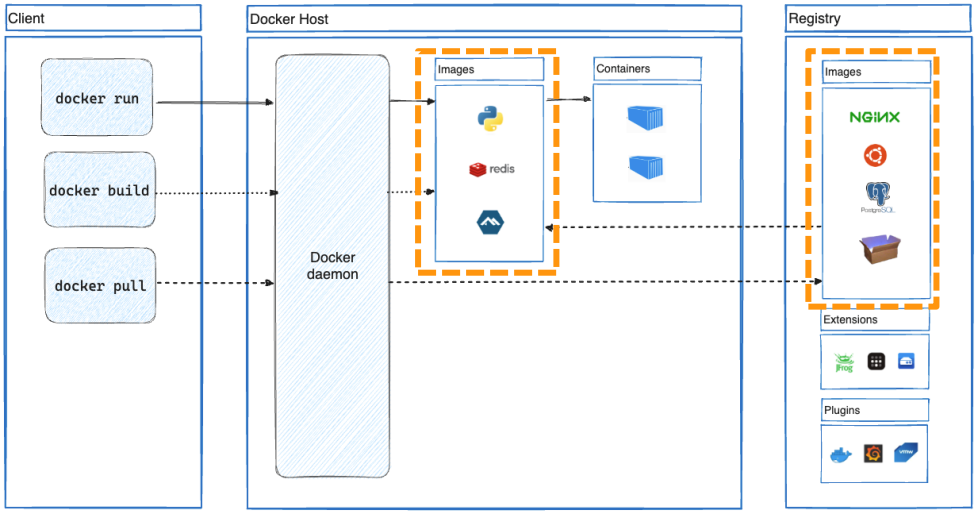
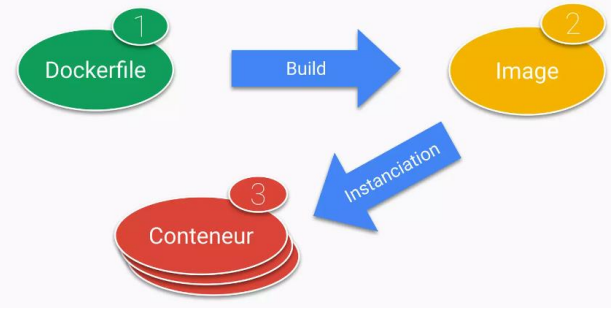
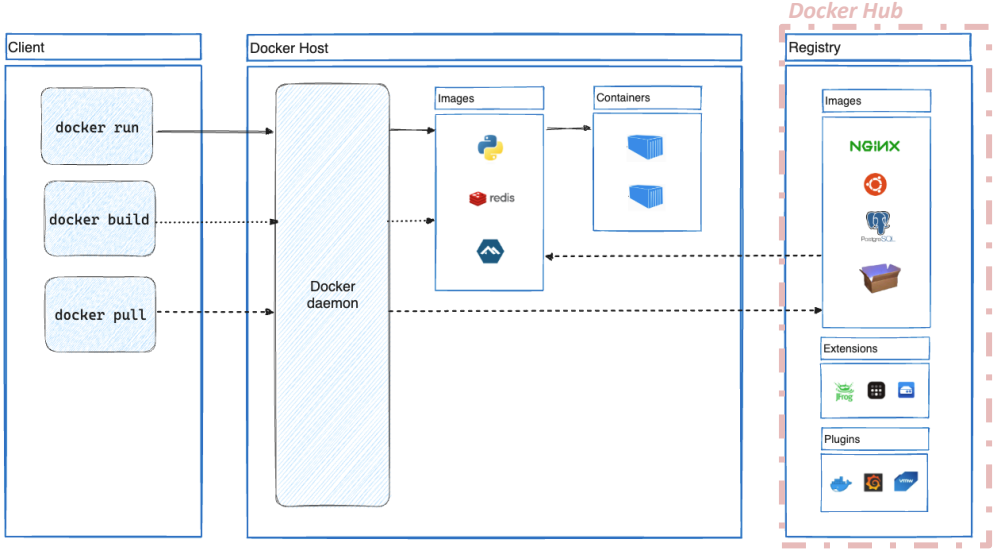


Image Docker

- Tout comme les machines virtuelles, les conteneurs Docker sont basés sur des images.
- Une image est un modèle en lecture seule qui contient toutes les instructions dont le moteur Docker a besoin afin de créer un conteneur Docker.
- Une image Docker est décrite sous la forme d'un fichier texte: **Dockerfile.**



Docker Hub

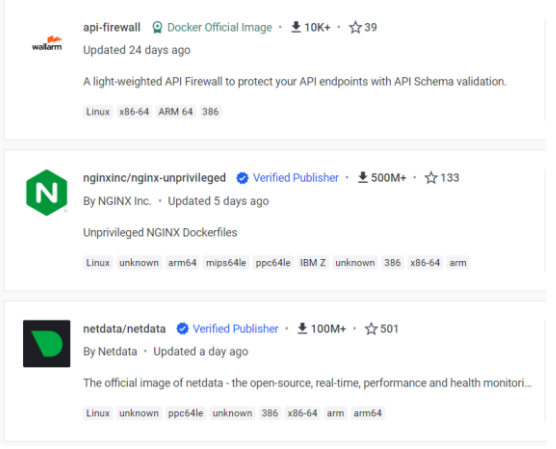


Docker Hub

- Le Docker Hub est un registre basé sur le Cloud, c’est une sorte de bibliothèque pour les images Docker.
  - un store ou les utilisateurs de Docker peuvent partager leurs images.
- Plusieurs images sont disponibles (des images de base et des images préconfigurées)
  - Les images de base officielles ont été créées par l'équipe de développement de Docker.
- Le service en ligne est divisé en un espace public et un espace privé
  - L’espace public offre aux utilisateurs la possibilité de télécharger leurs propres images et de les partager avec la communauté.
  - Les images téléchargées dans une zone privée du registre ne sont pas accessibles au public et peuvent, par exemple, être partagées au sein de l’entreprise ou avec des amis et des connaissances.
- Il est accessible via : https://hub.docker.com/

Docker Hub

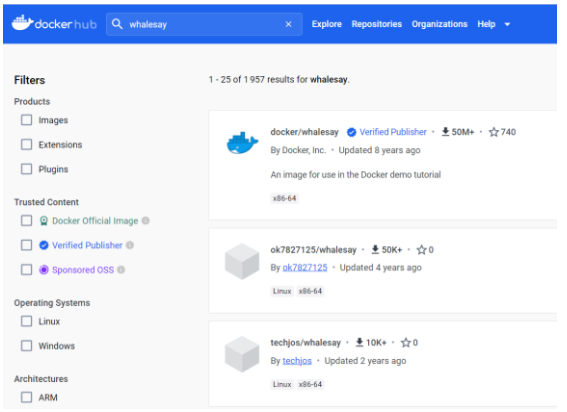
- Le nom de l'image est généralement sous la forme USER/IMAGE_NAME, sauf dans le cas d'une image officielle, où c'est seulement IMAGE_NAME
- Le nombre de stars : Le système de notation
- Le nombre de pulls : Le nombre de téléchargements

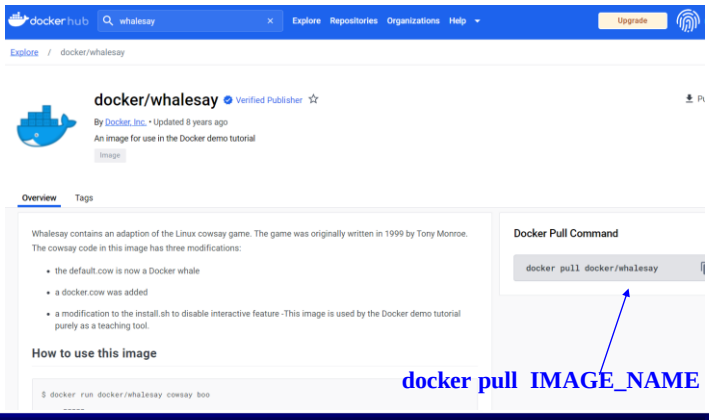


Gestion des Images Docker

- **Télécharger une image Docker** : commande docker pull IMAGE

Sur la page d'accueil de Docker Hub, on trouve par exemple l'image *whalesay* (docker demo) en tapant, on peux cliquer sur la ressource docker/whalesay pour accéder au Repository public de cette image.





Gestion des Images Docker

Afficher toutes les images du Docker sur votre système local

$ docker image ls

- La commande docker image donne tout sur les images locales, y compris la taille du fichier, la balise et l'ID de l'image

```
hp@hp-HP-Laptop-15-dw3xxx:~$ docker image ls
REPOSITORY        TAG       IMAGE ID       CREATED        SIZE
hello-world       latest    9c7a54a9a43c   6 months ago   13.3kB
docker/whalesay   latest    6b362a9f73eb   8 years ago    247MB
hp@hp-HP-Laptop-15-dw3xxx:~$
```

- REPOSITORY : nom de l'image.
- TAG : sont utilisés principalement pour affecter une version à une image
- ID : L'identifiant de l'image (unique pour chaque image téléchargée)
- CREATED : Date de la dernière modification de l'image
- SIZE : Taille de l'image

Gestion des Images Docker

Supprimer une image Docker

$ docker image rm [nom ou ID Image]:[tag]

- Une image Docker ne peut pas être supprimer si des conteneurs ont été instanciés depuis cette image (ça va aussi supprimer nos conteneurs qui se basent sur cette image
- On peut forcer la suppression pour éliminer aussi les conteneurs liés à l'image
- **$ docker image rm –f [nom ou ID Image]:[tag]**

Gestion des Conteneurs Docker

Démarrer une image Docker en tant que conteneur

$ docker run [OPTIONS] [nomImage]:[tag]

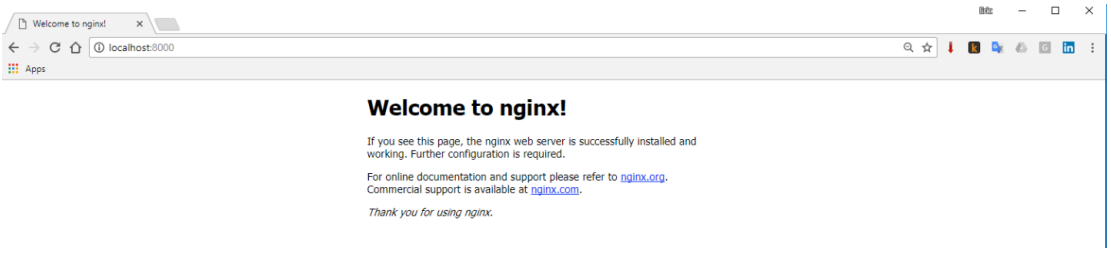
- La seule partie obligatoire est le nom de l'image Docker désiré
- Exemple d'option
  - **-d** (abréviation --detach) : pour exécuter le conteneur en arrière plan. Il permet de continuer à utiliser la console pendant que le conteneur tourne;
  - **-p** (abréviation --publish) pour définir l'utilisation de ports. La commande crée un mappage de port entre l'hôte et le conteneur.
    - Sans le mappage des ports, vous ne pourriez pas accéder à l'application depuis l'hôte

Gestion des Conteneurs Docker

Démarrer une image Docker en tant que conteneur

- Démarrons Nginx (serveur HTTP + reverse proxy) et exposons le port 80 du conteneur sur le port 8000 de notre machine:

  $ docker run -d -p 8000:80 nginx



Gestion des Images Docker

Démarrer une image Docker en tant que conteneurs

- Lorsque vous démarrez un conteneur, l'image sous-jacente est téléchargée sous forme de copie à partir du référentiel et stockée de façon permanente sur votre ordinateur.
- Un nouveau téléchargement n'est lancé que si la source de l'image change, par exemple si une version plus récente est disponible dans le référentiel.

```
hp@hp-HP-Laptop-15-dw3xxx:~/Bureau$ docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
578acb154839: Pull complete
e398db710407: Pull complete
85c41ebe6d66: Pull complete
7170a263b582: Pull complete
8f28d06e2e2e: Pull complete
6f837de2f887: Pull complete
c1dfc7e1671e: Pull complete
Digest: sha256:86e53c4c16a6a276b204b0fd3a8143d86547c967dc8258b3d47c3a2
Status: Downloaded newer image for nginx:latest
5e859f94fbf99c3e749104464f7a34fdab1e3aa63df7c51f2486c32efc579dfb
```

Gestion des Conteneurs Docker

Afficher tous les conteneurs exécutés sur le système

$ docker ps -a

- La sortie du terminal comprend des informations telles que l'ID du conteneur respectif, l'image sous-jacente, la commande exécutée lorsque le conteneur a été démarré, l'heure à laquelle le conteneur respectif a été démarré et l'état actuel.

```
hp@hp-HP-Laptop-15-dw3xxx:~/Bureau$ docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS
5e859f94fbf9   nginx             "/docker-entrypoint.…"   17 seconds ago  Up 15 seconds
f8214b28518f   getting-started   "docker-entrypoint.s…"   4 days ago      Created
f20f3e9b8fa6   hello-world       "/hello"                 5 days ago      Exited (0) 5 days ago
1b40e1b617de   c1087b2b41a9      "docker-entrypoint.s…"   3 months ago    Exited (0) 3 months ago
24c8eb35faa8   c1087b2b41a9      "docker-entrypoint.s…"   3 months ago    Created
485e5ec04644   hello-world       "/hello"                 3 months ago    Exited (0) 3 months ago
60af50e0659d   hello-world       "/hello"                 3 months ago    Exited (0) 3 months ago
```


Gestion des Conteneurs Docker

- Stopper un conteneur Docker

  $docker stop (Nom ou ID du conteneur)

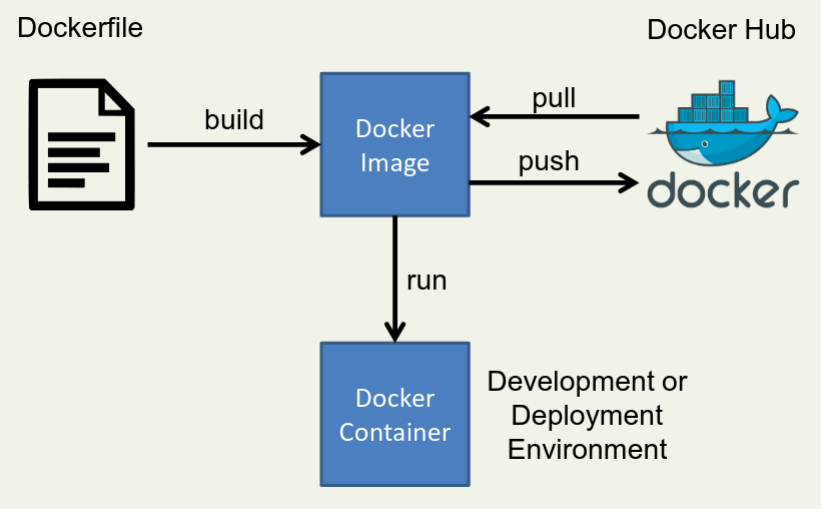
- Démarrer un conteneur stoppé

  $docker start (Nom ou ID du conteneur)

- Supprimer un conteneur stoppé

  $docker rm (Nom ou ID du conteneur)

Image Docker - Architecture



Dockerfile

- Le Dockerfile est un fichier qui contient toutes les instructions et arguments pour créer une image, comme les commandes à exécuter pour installer un logiciel, etc.
- Exemple d'un Dockerfile

Instruction	Argument

```
# Utiliser une image de base
FROM Ubuntu

# Définir le répertoire de travail
WORKDIR /app

# Mettre à jour le répertoire apt et Installer les dépendances
RUN apt-get update
RUN apt-get install python

RUN pip install flask
RUN pip install flask-mysql

# Copier le code de l'application
COPY . .

# Définir la commande de démarrage de l'application
CMD ["python", "app.py"]
```

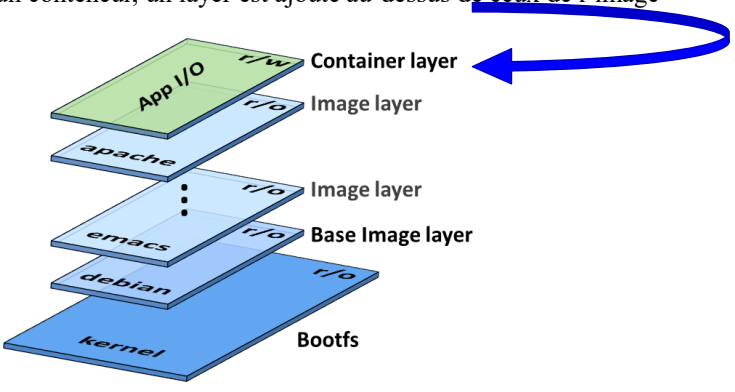
Dockerfile

Image de base ?

- Image préexistante qui sert de point de départ sur laquelle on peux ajouter des couches supplémentaires pour construire notre propre environnement d'exécution
  - Image *Ubuntu* : fournit un environnement Linux minimal basé sur la distribution Ubuntu
  - Image *alpine* : fournit une version très légère de Linux
  - Image *node* : configurée pour exécuter des applications Node.js
- Image de base → gain du temps car elle contient une configuration système ou en environnement logiciel minimal et on peux la personnaliser

Création d'une image Docker

- Une image docker est une succession de couches (layers) qui contiennent une liste de modifications du système de fichiers
  - On peut faire une analogie avec GIT où chaque layer serait un commit
- Au moment de la création d'un conteneur, un layer est ajouté au-dessus de ceux de l'image



Dockerfile

- Chaque **instruction** va créer une nouvelle **layer** correspondant à chaque **étape** de la construction de l'image, ou de la recette.

```
# Utiliser une image de base
FROM Ubuntu

# Définir le répertoire de travail
WORKDIR /app

# Mettre à jour le répertoire apt et Installer les dépendances
RUN apt-get update
RUN apt-get install python

RUN pip install flask
RUN pip install flask-mysql

# Copier le code de l'application
COPY . .

# Définir la commande de démarrage de l'application
CMD ["python", "app.py"]
```

Layer 1 : Base Ubuntu Layer

Layer 2 : Working directoty

Layer 3 : Changes in apt package

Layer 4 : Changes in pip package

Layer 5 : Source code

Layer 6 : Default command when running the container

Création de votre propre une image Docker

Création de votre propre Dockerfile

- Nous allons utiliser le code de l'application ''To Do'' (application Node.js) :

  https://github.com/docker/getting-started-app/tree/main

- Nous allons créer le Dockerfile et le mettre dans le répertoire racine du projet

```
#image source
FROM node:18-alpine

#définir le répertoire de travail
WORKDIR /app

# copier le code source
COPY . .

#installer les dépendances
RUN yarn install --production

#définir la commande par défaut lors de l'exécution de vos conteneurs Docker
CMD ["node", "src/index.js"]

#définir les ports d'écoute par défaut
EXPOSE 3000
```

RUN yarn install --production

Cette commande installe les dépendances de production de l'application Node.js. (les dépendances listées sous dans `package.json`)

→ yarn est un gestionnaire de paquets (similaire à npm)

L'option `--production` indique qu'il ne faut installer que les dépendances nécessaires au fonctionnement de l'application (excluant les dépendances de développement).

Création de votre propre une image Docker

Création de votre propre Dockerfile

- Création de l'image Docker

docker build -t getting-started .

L'option –t permet de marquer l'image

Signifie que le Dockerfile se trouve dans le répertoire courant

Création de votre propre une image Docker

Création de votre image Docker

docker build -t getting-started .

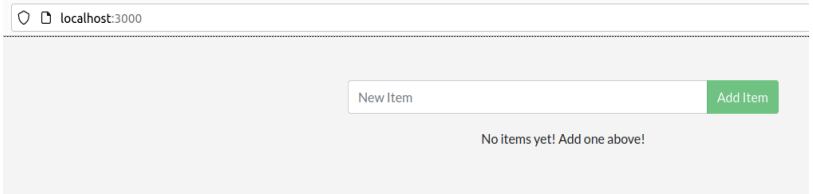
```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker build -t getting-started .
[+] Building 54.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 466B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/node:18-alpine
 => [auth] library/node:pull token for registry-1.docker.io
 => [1/4] FROM docker.io/library/node:18-alpine@sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d
 => => resolve docker.io/library/node:18-alpine@sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d
 => => sha256:d1517ab6615b781f3b81f339100063d1b2b41f1a32a9efb8563ecd1375311c22 6.78kB / 6.78kB
 => => sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa 3.40MB / 3.40MB
 => => sha256:3130715204cf4a9be94608d180505f50862416589a8f03eba7b664f15b9c0283 47.88MB / 47.88MB
 => => sha256:b06de8ab1c4feccaf7b687bb7ebd5180c2bd1f59d91749619d52af77fd38ea13 2.34MB / 2.34MB
 => => sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d 1.43kB / 1.43kB
 => => sha256:51490771aba658439d29b1b03b60fc31e67bf0da3e01cb5903716310df4be1c1 1.16kB / 1.16kB
 => => sha256:90ef3ffc51561ffa7dfafd2dc93f44601f8d4d4273ad8d54dbf34326c746a142 448B / 448B
 => => extracting sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa
 => => extracting sha256:3130715204cf4a9be94608d180505f50862416589a8f03eba7b664f15b9c0283
 => => extracting sha256:b06de8ab1c4feccaf7b687bb7ebd5180c2bd1f59d91749619d52af77fd38ea13
 => => extracting sha256:90ef3ffc51561ffa7dfafd2dc93f44601f8d4d4273ad8d54dbf34326c746a142
 => [internal] load build context
 => => transferring context: 4.59MB
 => [2/4] WORKDIR /app
 => [3/4] COPY . .
 => [4/4] RUN yarn install --production
 => exporting to image
 => => exporting layers
 => => writing image sha256:4ac73a4be5d6e114a9c8e7d6c8c2fc28002c4d98d0214a3ad3d43ed612f13e82
 => => naming to docker.io/library/getting-started
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Création de votre propre une image Docker

Démarrer votre Conteneur **docker run -dp 3000:3000 getting-started**

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS          PORTS                                      NAMES
64980821584e   getting-started  "docker-entrypoint.s…"   40 seconds ago   Up 39 seconds   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  affectionate_diffie
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Navigateur de la machine hôte :



Création de votre propre une image Docker

Mettre à jour votre application

- Faire une modification dans le fichier «src/static/js/app.js » (L56 par exemple, changer le message : « No items yet! Add one above!”)
- Effectuer un build pour la nouvelle version de l’image : docker build -t getting-started .
- Démarrez un nouveau conteneur en utilisant le code mis à jour :

docker run -dp 3000:3000 getting-started

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker run -dp 3000:3000 getting-started
f8214b28518f3a59ae6445dccf9e0e37e99a0c8e25159761ae104d236e74525e
docker: Error response from daemon: driver failed programming external connectivity on endpoint mystifying_mcnulty (481596
1282c3bec4307422e80dbf3ce0a541c14ca0e680e99bc8bed4f2531704): Bind for 0.0.0.0:3000 failed: port is already allocated.
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

- Une erreur s’affiche parce que l'ancien conteneur utilise déjà le port 3000 de l'hôte et un seul processus sur la machine (conteneurs inclus) peut écouter un port spécifique. Pour résoudre ce problème, vous devez supprimer l'ancien conteneur puis démarrer le conteneur de l’application mise à jour

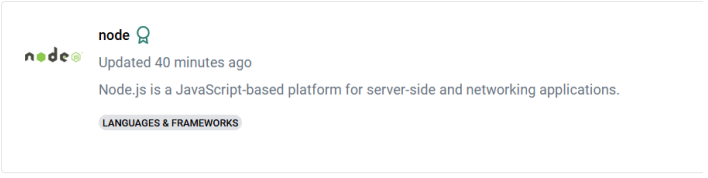
Création de votre propre une image Docker

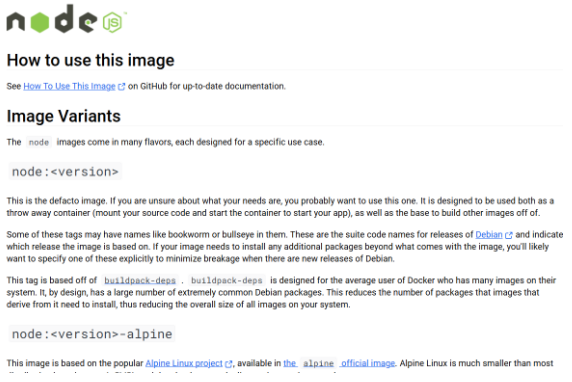
Mettre à jour votre application

- Question : est ce qu'il est possible de modifier le code et de voir automatiquement les changements sans avoir à recréer l'image
- Réponse : Oui! En utilisant les **Bind Mount** (ou mappage de répertoire) pour l'application
- Bind mount : permet de partager un répertoire du système de fichiers de l'hôte dans le conteneur
  - Pourra être utilisé pour monter le code source dans le conteneur. Le conteneur voit immédiatement les modifications que vous apportez au code, dès que vous enregistrez un fichier. Vous pouvez donc exécuter des processus dans le conteneur qui surveillent les modifications du système de fichiers et y répondent
  - Par exemple, pour une application Node.js, il y a un outil appelé *nodemon* qui permet de surveiller les modifications de fichiers, puis redémarrer automatiquement l'application.
  - Il existe des outils équivalents dans la plupart des autres langages et frameworks

Application

- Créer un `Dockerfile` pour une application `Node.js` dans un répertoire `my-app` avec les fichiers `index.js` (qui contient le code source de l'application) et `package.json` (pour les dépendances de l'application). L'application écoute sur le port `8080`.





Création de votre propre image Docker

Portabilité d'une image Docker

- Une image Docker construite sur un SE (Linux ou Windows) ne sera pas directement portable vers l'autre système sans quelques ajustements ou configurations spécifiques
  - Les images Docker sont construites à partir de couches qui utilisent un système de fichiers spécifiques à la platforme…

→ **Build « Multi-platformes »**

Création de votre propre image Docker

Build « Multi-platformes »

- Le moyen le plus simple de commencer à créer des images pour plusieurs plates-formes est d'utiliser l'émulation. Avec l'émulation, vous pouvez créer votre application sur plusieurs architectures sans avoir à apporter de modifications à votre Dockerfile.
- Il suffit de transmettre l'indicateur --platform à la commande build, en spécifiant le SE et l'architecture pour lesquels vous souhaitez créer l'image
- La commande ci-dessous crée l'image du serveur pour la plateforme Linux/arm/v7 :
  - docker build -t=server --platform=linux/arm/v7 .

Création de votre propre image Docker

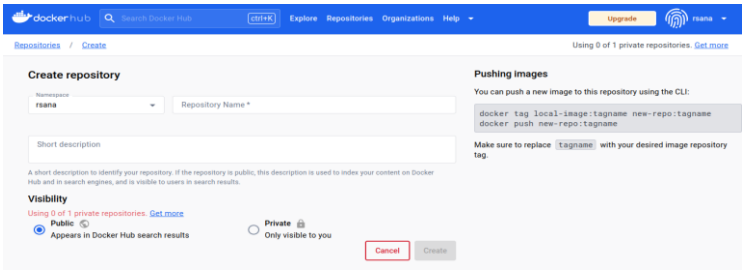
Build « Multi-platformes »

- Il est possible d'utiliser l'émulation pour produire des sorties pour plusieurs plates-formes à la fois.
  - Ex : --platform=linux/amd64,linux/arm/v7,linux/arm64/v8
- Mais, le pilote de build par défaut ne prend pas en charge les build multiplateformes simultanées. Donc, d'abord, on doit utiliser un autre générateur, qui utilise un pilote prenant en charge les versions multiplateformes simultanées
- Docker Buildx
  - Buildx est le client de build de nouvelle génération et il offre une expérience utilisateur similaire à la commande docker build classique, tout en prenant en charge des fonctionnalités supplémentaires.

Publier une image dans le Docker Hub

- Vous pouvez partager votre image auto-créée dans le Docker hub pour la rendre disponible à la communauté ou à un groupe de travail
- Un Docker ID est nécessaire pour accéder au Docker Hub
- Etapes nécessaires pour publier l'image:

Étape 1. Créer un repository : connectez-vous au Docker Hub avec votre ID Docker et votre mot de passe personnel et créez un repository public nommé getting-started (le même nom que l'image)



Publier une image dans le Docker Hub

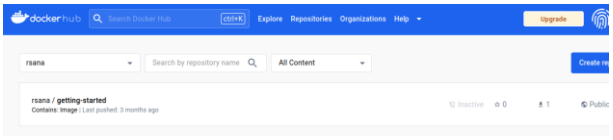
Étape 2. Tags Image, cette étape s'appelle le balisage (Tagging).

- Marquez l'image *getting-started* à l'aide de la commande docker tag dans la ligne de commande selon le schéma suivant :
  - $ docker tag [Image][Docker-ID]/[Image-Name]:[TAG]
  - Exemple: $ docker tag getting-started myUserName/getting-started:version1.0
- Vérifiez si l'image a été correctement balisée dans l'aperçu via la commande « docker image ls ». Le nom du référentiel devrait contenir votre identifiant Docker.

Publier une image dans le Docker Hub

Étape 3. Publier une Image

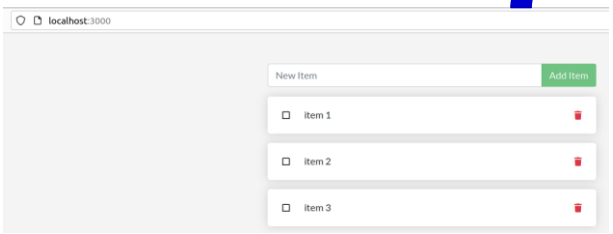
- Il faut se connecter au Docker Hub avec la commande docker login : $ sudo docker login
  - Le terminal vous invite à entrer votre nom d'utilisateur (Docker ID) et votre mot de passe.
- Si la connexion a réussi, utilisez la commande docker push pour publier votre image vers le Repository nouvellement créé.
  - $ sudo docker push myUserName/getting-started
- Connectez-vous au Docker-Hub via votre navigateur pour voir l'image téléchargée.

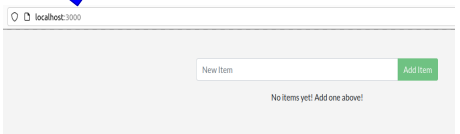


Persistance des données avec les volumes

- Pour cette application « To Do » (dont l'image Docker est intitulée « getting-started »), remarquez que votre liste de tâches est vide à chaque fois que vous lancez le conteneur

Redémarrage de l'instance du conteneur





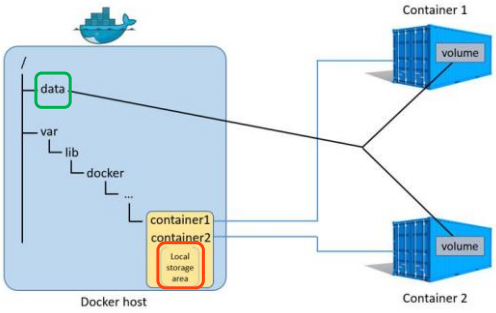
Persistance des données avec les volumes

- Par défaut, l'application « To Do » stocke ses données dans une base de données SQLite : un fichier todo.db dans le système de fichiers du conteneur
- Lorsqu'un conteneur est supprimé, toutes les données écrites dans le conteneur (c.à.d.. qui ne sont pas stockées dans un volume de données) sont supprimées avec le conteneur.

- En **créant un volume** et en **l'attachant** (souvent appelé « montage ») au **répertoire dans lequel vous avez stocké les données,** vous pouvez conserver les données. Au fur et à mesure que le conteneur écrit dans le fichier todo.db, il **conservera les données sur l'hôte dans le volume**.

Persistance des données avec les volumes

- Le schéma montre un seul hôte Docker exécutant deux conteneurs. Chaque conteneur existe à l'intérieur de son propre espace d'adresse dans la zone de stockage locale de l'hôte Docker (/ var / lib / docker / ...).
- Il existe également un volume de données partagé situé à / data sur l'hôte Docker.
- Les volumes de données résident à l'extérieur de la zone de stockage locale sur l'hôte Docker, renforçant ainsi leur indépendance. Lorsqu'un conteneur est supprimé, toutes les données stockées dans les volumes de données persistent sur l'hôte Docker.



Persistance des données avec les volumes

Créer un volume avec la commande : docker volume create todo-db

- Stopper et supprimer le conteneur de l'application qui fonctionne sans utiliser le volume persistant

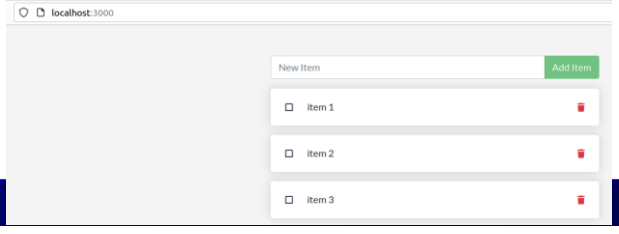
  docker rm -f <id-conteneur>

- Démarrer de nouveau le conteneur en ajoutant l'option **--mount** pour spécifier le volume

  Montez le volume par exemple sur /etc/todos dans le conteneur, ce qui permet de capturer tous les fichiers créés sur le chemin

  docker run -dp 3000:3000 --mount type=volume,src=todo-db,target=/etc/todos getting-started

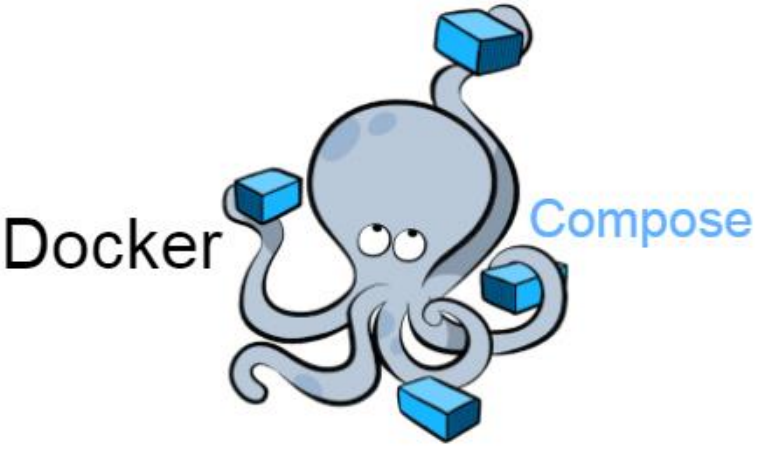
- Vérifier que les données persistent (en redémarrant et aussi en supprimant le conteneur)



Persistance des données avec les volumes

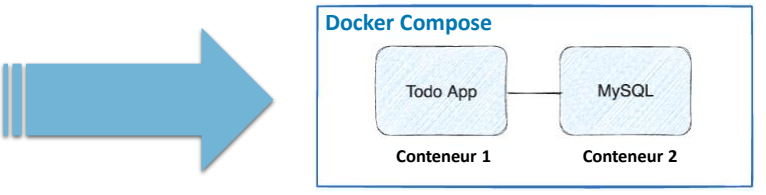
- Volume Docker : mécanisme pour stocker des données en dehors du système de fichiers du conteneur. Les données des volumes persistent même si le conteneur est supprimé
  - Séparation claire entre les données et l'application
  - Plusieurs conteneurs peuvent accéder au même volume simultanément.
- Docker gère entièrement le volume, y compris l'emplacement de stockage sur le disque (ils sont stockés dans /var/lib/docker/volumes/ (ou équivalent selon l'OS)). Il faut retenir seulement le nom du volume.
- Lister les volumes : `docker volume ls`
- Supprimer un volume : `docker volume rm nom_volume`

Gestion de plusieurs conteneurs avec Docker Compose



Docker Compose

- Supposons qu'on veut ajouter une base de données MySQL à la pile de notre application « To Do »
- Question : est ce qu'on installe MySQL dans le même conteneur ou on l'exécute séparément
  - Réponse : en général, chaque conteneur doit faire une seule chose et bien la faire
- Dans la vision Docker, un conteneur ne doit faire tourner qu'un seul processus



Docker Compose

Défis

- Applications multi-conteneurs: une application Docker qui inclut plusieurs conteneurs (par exemple, un serveur Web et une base de données s'exécutant dans des conteneurs distincts)
- La construction, l'exécution et la connexion des conteneurs à partir de fichiers Docker distincts sont fastidieuses

➢ **Docker Compose** un outil permettant de définir (dans un fichier YAML), et de gérer (en ligne de commande) plusieurs conteneurs comme un ensemble de services interconnectés

- Fichier YAML déclaratif: docker-compose.yml (placé à la racine du projet)
- exécuter une seule commande pour créer, exécuter et configurer tous les conteneurs décrits dans le fichier YAML.

Docker Compose

Les **principals sections** d'un fichier docker-compose.yml :

- **services** (les conteneurs à exécuter) : cette section définit les services qui composent l'application. Un service peut être une application, une base de données, etc. Chaque service est configuré pour être exécuté dans un conteneur Docker.

```
services:
  app:
    image: node:18-alpine
    ports:
      - "3000:3000"
```

Le nom du premier service

- **volumes** : utilisés pour persister les données entre les redémarrages des conteneurs ou pour partager des données entre plusieurs services.
- **networks** : permettent aux conteneurs de communiquer entre eux. Docker Compose crée par défaut un réseau, mais on peux définir un ou plusieurs réseaux en cas de besoin.

Docker Compose

- Après avoir définir un volume dans la section « volumes » (même niveau que les services), on peux utiliser ce volume dans un service :

```
services:
  app:
    image: node:18-alpine
    ports:
      - "3000:3000"
  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data :/var/lib/mysql

volumes:
  todo-mysql-data :
```

- De même, une fois un réseau est définit dans la section « networks » (même niveau que les services), on peux lier les services a ce réseau :

```
services:
  app:
    image: node:18-alpine
    networks:
      - mynetwork
  mysql:
    image: mysql:8.0
    networks:
      - mynetwork
networks:
  mynetwork:
```

Docker Compose : Exemple

```
version: "3.8"          ← Section 1

services:               ← Section 2
  app:                  ← Service 1
    image: node:18-alpine
    working_dir: /app
    ports:
      - "3000:3000"
    environment:
      - MYSQL_HOST=mysql
      - MYSQL_USER=root
      - MYSQL_PASSWORD=my-secret-pw
      - MYSQL_DATABASE=todo_db
```

(Suite)

```
  mysql:                ← Service 2
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: my-secret-pw
      MYSQL_DATABASE: todo_db  # Création de la base de
données au démarrage
    volumes:
      - todo-mysql-data:/var/lib/mysql

volumes:                ← Section 3
  todo-mysql-data:
```


Docker Compose : Exemple

L'image Mysql dispose de plusieurs variables d'environnements

Pour en savoir plus sur les variables d'environnement : Mysql Docker Hub listing

Environment Variables

When you start the `mysql` image, you can adjust the configuration of the MySQL instance by passing one or more environment variables on the `docker run` command line. Do note that none of the variables below will have any effect if you start the container with a data directory that already contains a database: any pre-existing database will always be left untouched on container startup.

See also https://dev.mysql.com/doc/refman/5.7/en/environment-variables.html for documentation of environment variables which MySQL itself respects (especially variables like `MYSQL_HOST`, which is known to cause issues when used with this image).

`MYSQL_ROOT_PASSWORD`

This variable is mandatory and specifies the password that will be set for the MySQL `root` superuser account. In the above example, it was set to `my-secret-pw`.

`MYSQL_DATABASE`

This variable is optional and allows you to specify the name of a database to be created on image startup. If a user/password was supplied (see below) then that user will be granted superuser access (corresponding to `GRANT ALL`) to this database.

`MYSQL_USER`, `MYSQL_PASSWORD`

These variables are optional, used in conjunction to create a new user and to set that user's password. This user will

Lancer la stack Docker Compose

Démarrer l'ensemble des conteneurs en arrière plan

docker compose up -d

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker compose up -d
[+] Running 17/17
 ✔ mysql 11 layers [⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿]      0B/0B      Pulled
   ✔ 8e0176adc18c Pull complete
   ✔ a6b6bf6e5d0f Pull complete
   ✔ c17b83f8620f Pull complete
   ✔ b2e259cd9b6c Pull complete
   ✔ 366131ab00d1 Pull complete
   ✔ 2f99ba83a3cb Pull complete
   ✔ f7c88955f01f Pull complete
   ✔ 577fb415d7f8 Pull complete
   ✔ 29160ed46eb1 Pull complete
   ✔ 69ce9884ce5d Pull complete
   ✔ 848f0dceb14c Pull complete
 ✔ app 4 layers [⣿⣿⣿⣿]      0B/0B      Pulled
   ✔ 96526aa774ef Already exists
   ✔ 3130715204cf Already exists
   ✔ b06de8ab1c4f Already exists
   ✔ 90ef3ffc5156 Already exists
[+] Running 4/4
 ✔ Network getting-started-app-main_default          Created
 ✔ Volume "getting-started-app-main_todo-mysql-data"  Created
 ✔ Container getting-started-app-main-mysql-1        Started
 ✔ Container getting-started-app-main-app-1          Started
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

docker ps

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
81c86c760193   node:18-alpine "docker-entrypoint.s…"   8 seconds ago  Up 6 seconds  127.0.0.1:3000->3000/tcp getting-started-app-main-app-1
216dbd9632b6   mysql:8.0      "docker-entrypoint.s…"   8 seconds ago  Up 6 seconds  3306/tcp, 33060/tcp      getting-started-app-main-mysql-1
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```


Gestion de plusieurs conteneurs avec Docker Compose

Détruire l'ensemble des ressources d'une stack:

docker compose down

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker compose down
[+] Running 3/3
 ✔ Container getting-started-app-main-mysql-1  Removed
 ✔ Container getting-started-app-main-app-1    Removed
 ✔ Network getting-started-app-main_default    Removed
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Conclusion

- Dans ce chapitre
  - Fonctionnement d'un conteneur Docker
  - Création d'une image Docker : fichier Dockerfile
  - Gestion de plusieurs conteneurs avec Docker Compose
- Les conteneurs : **évolutivité**, **flexibilité** et **facilité**
  - Pour faire évoluer les fonctionnalités de l'application, il suffit alors de déployer de nouveaux conteneurs renfermant les composants appropriés (au lieu de créer entièrement de nouvelles itérations de l'application)

Conclusion

- Les conteneurs : solution idéale pour le développement d'applications de type microservices
  - permettent de configurer des applications complexes à partir de composants élémentaires
  - chacun de ces composants peut être déployé dans un conteneur et les conteneurs constitutifs étant reliés entre eux pour former l'application complète
- Mais, les conteneurs **ne conviennent pas à toutes les applications**
- Ils ne peuvent pas remplacer tous les déploiements de VM existants
- Les conteneurs : <u>ne sont pas adéquats pour les applications monolithiques</u>
- Ces applications peuvent toujours s'exécuter comme des VMs entièrement fonctionnelles adossées à un hyperviseur conventionnel